TOPOLOGY / UNKNOT RECOGNITION

Matroidal Dual-Forest Selection for Unknot Recognition

Polynomial pair-column cancellation, exact deletion deficits, and a fixed-parameter higher-support kernel

Research contribution prepared with ChatGPT
for Vladimir Reshetnikov's *ProveIt* project
12 October 2026

Abstract

The maintained ProveIt normal-cut pipeline can eliminate all source core three-handles by a boundary-rooted dual forest. Its own experiments show that choosing an arbitrary such forest can leave an uncancellable raw core even when a different forest succeeds. We study the exact selection problem, with the original zero/one-handle forest fixed and with unsigned source incidences retained.

For matrices whose usable columns meet at most two rows, deletion closure is directed reachability. A successful dual tree must meet every unanchored source strongly connected component. We reduce that requirement to graphic/partition matroid intersection and derive the exact optimum

$$\delta = \max_{J \subseteq [q]} \left(|J| - \operatorname{rank}_D \left(\bigcup_{j \in J} C_j \right) \right)$$

for additional, formally deferred row deletions after the best tree. Both a completing schedule and a rank certificate of the optimum are obtained in polynomial time.

For general matrices, let b be the number of rows incident to usable columns of support at least three. Ordering those rows and grouping shared seed witnesses gives an exact algorithm with at most $b!\,\mathrm{Bell}(b)$ matroid-intersection subproblems, hence running time $\mathrm{poly}(\Lambda)2^{O(b\log(b+1))}$ for encoded input size Λ. An explicit satisfiability reduction proves that the unrestricted abstract problem is NP-complete even when every column has support at most three and all entries lie in $\{0, 1, 2\}$.

The package includes conventional proofs, an independent certificate verifier, 54 passing unit tests, exhaustive and randomized comparisons, a reconstruction of the repository's five-row loop example, and a source-replay integration seam. These are local selection results, not a general quasi-polynomial unknot recognizer. The hardness construction is not asserted to be geometrically realizable, and full native geometry and recognition were not executed in this study.

Status. Mathematical proofs and executed research kernels; not externally refereed or proof-assistant formalized. Historical priority is not claimed. Two potentially overlapping incoming dual-forest archives were inventoried but their contents were not audited.

Contents

1 Research position and the exact contribution **1**
1.1 The checkpoint being continued . . . 1
1.2 Relation to the quasi-polynomial target . . . 1
1.3 Attribution and novelty boundary . . . 1

2 Authenticated cores and the cancellation model **2**
2.1 Input, conventions, and scope . . . 2
2.2 The geometric interface inherited from the repository . . . 2
2.3 Why full positive cancellation is a peeling question . . . 3

3 Pair columns become directed reachability **4**
3.1 Usable columns and the dependency digraph . . . 4
3.2 Only unanchored source components require seeds . . . 4

4 Independent representatives with checkable rank obstructions **5**
4.1 The common combinatorial primitive . . . 5
4.2 Shortest augmentations in the graphic/partition instance . . . 5
4.3 Why an unsuccessful search proves optimality . . . 6

5 Polynomial forest choice and the exact deletion deficit **7**
5.1 An optimum, not only a yes/no test . . . 7

6 A fixed-parameter extension for higher-support columns **8**
6.1 The exceptional row boundary . . . 8
6.2 Why ordering only exceptional rows is complete . . . 9
6.3 Grouping witnesses rather than guessing their row names . . . 10
6.4 The algorithm and its bit bound . . . 10

7 A sharp abstract hardness boundary **11**

8 The native loop matrix, worked explicitly **12**

9 Certificates, resource semantics, and integration **13**
9.1 Positive packets . . . 13
9.2 Optimality and negative packets . . . 13
9.3 Unknown is distinct from no . . . 14
9.4 The native seam and what has not been executed . . . 14

10 Reproducible validation and measured tradeoffs **15**
10.1 Executed checks . . . 15
10.2 A family with exponentially many dual trees . . . 15
10.3 Paired matrix-stage measurements . . . 16

11 What this proves toward quasi-polynomial recognition **16**

12 Further research questions and concrete next steps **17**

13 Conclusion **19**

A Algorithm pseudocode and certificate interfaces **19**
A.1 The pair-column selector . . . 19
A.2 The general selector . . . 20
A.3 Reproduction commands . . . 20

B Source and artifact provenance **20**

1 Research position and the exact contribution

1.1 The checkpoint being continued

This article is pinned to ProveIt commit

`c6d0ec8aa0b6308310ab6c081ba29d8bc363ede5`.

The immediate inputs are the maintained `synthesis/column_cancellation.tex`, `synthesis/core_three_handles.tex`, and `synthesis/three_handles_results.tex`, together with the corresponding native producer and source checker [1, 2, 3, 4, 5]. The relevant distinction is between *eliminating all three-handles* and *choosing three-handle cancellations that leave a useful lower presentation*. The first already has a complete sparse source-bound procedure. The second was explicitly left open in the inspected maintained discussion.

The recorded loop-source experiments are unusually useful: they identify a failure caused by a choice, rather than by the absence of a certificate. Four possible first dual edges are eligible; the default edge leaves a stopping set, while each of the other three permits completion. Enumerating all spanning trees is a valid finite fallback, but its worst-case size can be exponential. Our goal is to make this selection step exact without paying for every tree on a structurally broad class, and to isolate what must be small outside that class.

The contribution is not a new test based on signed exponent sums. The source matrix records *unsigned geometric intersections*; the value two means at least two intersections, not zero modulo two. Source topology remains a separate input obligation. In particular, the same matrix-and-graph optimization may be applied to many abstract instances which need not arise from an actual normal cut.

1.2 Relation to the quasi-polynomial target

The specific target displayed in Lackenby's inspected lecture slides is $n^{O(\log n)}$, for n crossings [6]. The July 2026 preprint *Incompressible surfaces, hierarchies and unknot recognition* supplies a hierarchy framework; its Section 9 distinguishes a bound on iterations from the still unspecified costs of some operations [7]. The present paper therefore charges its local search explicitly, without inferring a recognition bound merely from a short hierarchy or a compressed cut.

We prove a polynomial special case, a fixed-parameter generalization, and a hardness boundary for the abstract selector. Section 11 states a conditional composition theorem specifying exactly which further bounds would make this component compatible with the stated target. It does not assume that those further bounds are already true.

1.3 Attribution and novelty boundary

Stopping-set persistence and complete full-row peeling are maintained results, recalled in Section 2, not new claims. Rooted dual two/three cancellation is also inherited. The graphical matroid, independent representatives, and shortest-augmenting-path framework are classical; a modern primary reference that recalls the relevant intersection machinery is [8].

What is developed here is the directed reduction for unsigned pair-column cores, the resulting exact deletion-deficit interpretation, the exceptional-row ordering and shared-witness partition reduction, and an explicit support-three hardness construction for the combined problem. The proofs are

given independently of experimental success. We do not assert first priority in the wider literature. Moreover, `docs/incoming` contained two recently delivered archives with dual-forest titles whose binary bodies were not inspected. This is a real audit limitation, not evidence that those archives lack overlapping results. The package's `SOURCE_AUDIT.md` names them and separates their inventory from the maintained text actually read.

2 Authenticated cores and the cancellation model

2.1 Input, conventions, and scope

Let $E = \{0, \dots, m-1\}$ be a row set, and let

$$N = (n_{rh})_{r \in E,\, h \in H} \in \mathbb{N}^{E \times H}$$

be a finite nonnegative matrix. Let $D = (V, E)$ be a connected undirected multigraph with a distinguished vertex ∂. A row labels one dual edge, including possibly a loop. Parallel edges are distinct. Put $v = |V|$ and let s be the number of nonzero source entries. The input length Λ includes all row, column, and endpoint labels and all coefficient bits. Zero columns can be omitted.

For the intended native application, $V \setminus \{\partial\}$ consists of core three-handles, and a row is a source two-handle. Original-boundary and cut-boundary endpoints are identified with ∂. The matrix is formed after one fixed zero/one-handle forest has been contracted. Its columns are the retained one-handles, and its entries count intersections of attaching curves with their belt disks. The source must authenticate these facts; arbitrary group relators are not substitutes for the geometric counts.

Definition 2.1 (Dual-forest cancellation selection)**.** A spanning tree T of D is *successful* if $N[E \setminus T, H]$ admits complete positive one/two cancellation. The selection problem asks whether such a tree exists and, on success, asks for a tree and a complete schedule.

A "no" answer in Definition 2.1 means only that this fixed presentation and move family cannot finish. It is not a certificate that a manifold is not a handlebody, or that a knot is nontrivial. A different primal forest, geometric intersection reduction, or new cutting surface may change the answer.

2.2 The geometric interface inherited from the repository

Proposition 2.2 (Source-bound dual-tree interface; inherited)**.** *Assume the authenticated core charts satisfy the endpoint description in [2]. Every spanning tree of D can be canceled in root-to-leaf order by geometric two/three-handle cancellations. The surviving two-handle attaching maps in the original zero/one body, and the bundle port charts in that body, are unchanged. Thus its effect on the fixed raw matrix is precisely deletion of the tree rows.*

Proof. For a tree edge from the already exposed root component to a new vertex, its two endpoint occurrences include exactly one live three-handle. It therefore supplies the authenticated single-intersection cancellation. Exposing the new vertex makes it part of the root component for subsequent steps. Repeating along the tree removes all $v - 1$ interior vertices. A loop has two equal live occurrences or no live occurrence, and never supplies a tree edge. Preservation of the lower body and ports is the geometric relative-cancellation statement established in the maintained source argument [2]; it is an assumption about those charts, not a conclusion from an arbitrary matrix. □

The source endpoint rule is especially transparent on an original edge with normal weight w. For $w = 0$, its gap has the two original vertex endpoints. For $w > 0$, the two extreme gaps each join one original endpoint to the cut boundary. This accounts for both repeated endpoints and edges directly to the boundary. The native independent replayer reconstructs this information from link triangles and local normal coordinates, rather than trusting the producer's graph [5].

2.3 Why full positive cancellation is a peeling question

A positive pivot at (r, g) has $n_{rg} = 1$. After deleting its row and column, the other unsigned incidences obey

$$n'_{ih} = n_{ih} + n_{ig}n_{rh}. \tag{1}$$

A *unit-column peel* is the case where column g has total one on all live rows. It creates no fill in surviving rows.

Definition 2.3 (Stopping set)**.** A nonempty $R \subseteq E$ is a stopping set if, for every column h,

$$S_h(R) := \sum_{r \in R} n_{rh} \quad \text{is either zero or at least two.}$$

Lemma 2.4 (Stopping persistence; inherited)**.** *A nonempty stopping set prevents full positive cancellation, even if nonzero-fill pivots are allowed. Maximal unit-column peeling deletes all rows if and only if full positive cancellation is possible. Its residual is the unique largest source stopping set.*

Proof. We reproduce the maintained argument [1] to identify the exact move system. If a pivot row r is outside R, then for a surviving column h,

$$S'_h(R) = S_h(R) + S_g(R)n_{rh}.$$

The right side is zero or at least two. If $r \in R$, then $S_g(R) \geq 2$, so $R' = R \setminus \{r\}$ is nonempty, and

$$S'_h(R') = S_h(R) + (S_g(R) - 2)n_{rh}.$$

When $S_h(R) = 0$, also $n_{rh} = 0$; otherwise the displayed sum is at least two. A nonempty stopping set therefore survives every positive-pivot prefix.

Peeling creates no fill. If it stalls on R, every surviving column has total zero or at least two on R. Every previously deleted column is zero on R, because it met only its deleted pivot row when used. Thus R is a source stopping set. Conversely, a peel cannot remove the first row of any source stopping set. Finally, a union of stopping sets is a stopping set: a column meeting the union already meets a constituent with total at least two. All source stopping sets lie in the residual, proving maximality and uniqueness. □

Lemma 2.5 (Monotone seed deletion and saturation)**.** *If deleting $F \subseteq E$ permits full peeling, deleting any $F' \supseteq F$ also does. Replacing each coefficient by* $\min(2, n_{rh})$ *preserves every peeling answer and, consequently, every full positive-cancellation answer.*

Proof. A nonempty stopping set contained in $E \setminus F'$ would also be a source stopping set contained in $E \setminus F$, contradicting Lemma 2.4. For saturation, the predicate that a nonnegative column sum is one depends only on which entries are zero, which are one, and which exceed one. Every successive static peel is therefore unchanged. Apply Lemma 2.4 again. □

It follows that a successful *independent* dual edge set suffices: extend it to a spanning tree, and the extra deletions cannot spoil success. This elementary observation is what connects cancellation closure to a matroid constraint.

3 Pair columns become directed reachability

3.1 Usable columns and the dependency digraph

A column is *usable* if at least one of its entries is one. A column with no unit entry can never be a static peeling pivot, regardless of how many other rows are deleted. It may therefore be omitted from the closure calculation.

Initially suppose every usable column has support size at most two. Construct a digraph G_N on the row set E and an anchor set A as follows. A singleton usable column on row r places r in A. For a column supported on $\{u, w\}$, put an arc $w \to u$ when $n_{uh} = 1$, and put an arc $u \to w$ when $n_{wh} = 1$. A pair with two unit entries gives two opposite arcs. A pair with entries one and two gives just the arc from the double-incidence row to the unit-incidence row.

For $S \subseteq E$, write $\mathrm{Reach}(S)$ for reachability in G_N, including S itself. Let $\mathcal{P}(F)$ denote all rows deleted by first deleting F and then peeling to exhaustion, including the initial rows F.

Theorem 3.1 (Exact directed closure)**.** *For a pair-column input and every initial deletion set F,*

$$\mathcal{P}(F) = \mathrm{Reach}(A \cup F). \tag{2}$$

Proof. A usable singleton column removes its anchored row unless that row is already gone. A usable pair column can remove a row u exactly when its other row has already been removed and its entry at u is one. These are precisely the anchor and directed-arc rules. Thus each actual peel stays in the right side of (2). Conversely, follow any directed path starting at an anchor or an initially deleted row. Once the predecessor is deleted, the column supplying the next arc removes its unit row if that row is still present. Induction on path length proves the reverse inclusion.

Deleting a pivot column does not invalidate this reasoning. When it is used, all its other incident rows are already gone; it has no remaining target whose future deletion might depend on reusing it. □

3.2 Only unanchored source components require seeds

Contract the strongly connected components of G_N to obtain its condensation DAG. A *source component* has no incoming arc from another component. Let

$$C_1, \ldots, C_q$$

be the source components containing no anchor. These row sets are disjoint.

Corollary 3.2 (Necessary and sufficient seed blocks)**.** *For a pair-column input, deleting F permits full cancellation if and only if*

$$F \cap C_i \neq \varnothing \qquad (1 \le i \le q). \tag{3}$$

Proof. An unanchored source component cannot be reached from another component, so it needs an initial seed in itself. Conversely, every component in a finite DAG is reachable from a source. A

source containing an anchor is already reached; a source meeting F is reached as well. Equation (2) completes the argument. □

Example 3.3 (The direction matters)**.** For the one-column matrix $(1,2)^{\mathsf{T}}$, the only arc is $1 \to 0$. Deleting row 1 clears the matrix, while deleting row 0 leaves the entry two. Replacing this digraph by an undirected support graph would incorrectly make both seeds equivalent. An anchor farther downstream also does not seed an unanchored source: an arc $0 \to 1$ and an anchor at 1 still require a seed at 0.

Remark 3.4 (This is support size, not matrix rank)*.* The condition concerns the number of rows touched by each usable column. It is unrelated to the rank of N over a field. In particular, no reduction of its unsigned coefficients modulo two is performed.

4 Independent representatives with checkable rank obstructions

4.1 The common combinatorial primitive

We need a primitive slightly more general than the disjoint blocks above. Let $Q_1, \dots, Q_k \subseteq E$ be arbitrary candidate sets. A partial independent transversal consists of distinct representatives, one for each of some colors, whose dual edges form a forest. Let $\nu(Q)$ be the maximum number of represented colors. For $X \subseteq E$, the graphic rank is

$$r_D(X) = v - \mathrm{cc}(V, X). \tag{4}$$

Loops contribute no rank; parallel edges cannot both belong to an independent set.

Introduce one *clone* (r, i) for each $r \in Q_i$. Clones of a row share its dual endpoints and are parallel in the graphic matroid. A partition matroid permits at most one clone of each color i. Consequently a common independent set is exactly a partial independent transversal. The number of clones is $P = \sum_i |Q_i|$.

Theorem 4.1 (Independent representatives and exact deficiency)**.** *There is a polynomial algorithm returning a maximum partial independent transversal and a color subset $J \subseteq [k]$ such that*

$$k - \nu(Q) = |J| - r_D\left(\bigcup_{i \in J} Q_i\right) = \max_{K \subseteq [k]} \left(|K| - r_D\left(\bigcup_{i \in K} Q_i\right)\right). \tag{5}$$

In particular, a full independent transversal exists if and only if every color subset K satisfies the corresponding rank inequality.

This is the classical independent-representative/matroid-intersection conclusion. We give its specialized algorithm and certificate proof so that the delivered checker does not need to trust an optimizer.

4.2 Shortest augmentations in the graphic/partition instance

Maintain a selected clone set I which is both a forest and color-distinct. An unselected clone x is a source if it can be added to the forest, and a sink if its color is unused. For selected y and unselected x, put an arc $y \to x$ when $I - y + x$ is a forest. Put an arc $x \to y$ when y occupies the

color of x. For a source, incoming arcs can be omitted; for a sink, outgoing arcs can be omitted. Neither omission changes a shortest source-to-sink path.

If the endpoints of x are already connected in I, the selected edges y allowing $y \to x$ are exactly the edges on their unique forest path. A loop has an empty path and is never addable. Thus all exchanges are found with ordinary forest traversals. A shortest source-to-sink path is found by breadth-first search. Toggle its selected and unselected vertices.

Lemma 4.2 (A shortest toggle is an augmentation)**.** *The toggle of a shortest source-to-sink path preserves the forest and color conditions and increases $|I|$ by one.*

Proof. Write the path as

$$x_0, y_1, x_1, \ldots, y_t, x_t,$$

where the x_i are unselected. For $i < t$, the color of x_i is the color of y_{i+1}; the terminal color of x_t is unused. All these occupied colors are distinct because the selected vertices of a shortest path are distinct. Deleting the y_i and inserting the x_i therefore preserves the color condition.

For the graphic condition, x_0 joins two different components of the original forest. Each x_i with $i \geq 1$ joins vertices in the same original component; otherwise it would itself be a source and shorten the path. Its fundamental cycle in $I + x_i$ contains y_i. It contains no y_j with $j < i$, since the arc $y_j \to x_i$ would shorten the chosen path. The matrix of these fundamental cycles on the deleted edges $(y_1, \ldots, y_t)$ is triangular over $\mathbb{F}_2$ with unit diagonal. It is therefore nonsingular.

All cycles in $I + \{x_0, \ldots, x_t\}$ are combinations of the fundamental cycles of $x_1, \ldots, x_t$: x_0 is the only added edge joining its original components and is not in a cycle. A nonzero combination cannot be zero on all the y_i by the triangularity just proved. Hence after those edges are deleted no cycle survives. The net increase in cardinality is one. The use of $\mathbb{F}_2$ here concerns graph cycle space only, not the source incidence matrix N. □

4.3 Why an unsuccessful search proves optimality

Proof of Theorem 4.1. Repeat Lemma 4.2 until there is no augmenting path. Let R be the set of clone vertices reached from all sources in the last exchange search. Every reached unselected clone has an occupied color whose selected clone is also reached, since no sink is reached. Thus the number of colors appearing in R is $|I \cap R|$.

Every unselected clone outside R is spanned by $I \setminus R$: it is not a source, and its fundamental forest path contains no selected edge in R, or the exchange arc would make it reached. Therefore the graphic rank of all clones outside R is $|I \setminus R|$.

Take J to be the colors not appearing in R. All clones of these colors are outside R, and hence

$$|J| - r_D\left(\bigcup_{i \in J} Q_i\right) \geq k - |I \cap R| - |I \setminus R| = k - |I|.$$

For any color subset K, the selected transversal contains at least $|I| - (k - |K|)$ independent representatives in its union. Consequently

$$|K| - r_D\left(\bigcup_{i \in K} Q_i\right) \leq k - |I|.$$

Equality holds for J. The same counting argument applied to any proposed transversal bounds its size by $k - |J| + r_D(\bigcup_{i \in J} Q_i) = |I|$. This proves optimality and the displayed min–max equality. □

The returned certificate needs only the colors J. A checker forms their union and counts graph components to verify the rank. It does not reproduce the augmenting paths or trust a matrix-rank computation. A positive partial transversal is checked simply for membership, distinct colors, and acyclicity.

Proposition 4.3 (A conservative explicit algorithmic bound)**.** *After dense index normalization, the above forest-path implementation uses $O(P(v+1)^2 + k + v)$ structural work, apart from input preparation, and polynomial storage. All operations involve labels or counts of polynomial bit length.*

Proof. There are at most $v-1$ augmentations. In a search, each of P clones requires at most an $O(v)$ forest traversal, and the resulting exchange graph has $O(Pv)$ arcs. Its breadth-first search takes the same order of work. Summing over at most v searches gives the bound. The degenerate graph $v = 1$ has no augmentations; its clones are all loops. Empty color classes are retained when forming the obstruction. Standard direct arrays after relabeling give the structural model; the supplied readable Python implementation uses dictionaries and sets, so individual hash-lookup timings are not being asserted as adversarial constant bit costs. The resulting worst-case bit cost is still polynomial. □

5 Polynomial forest choice and the exact deletion deficit

Theorem 5.1 (Polynomial pair-column selector)**.** *Suppose every usable column of N has support at most two. Let $C_1, \dots, C_q$ be the unanchored source components of G_N. A successful dual tree exists if and only if*

$$r_D\left(\bigcup_{i\in J} C_i\right) \geq |J| \qquad \text{for every } J \subseteq [q]. \tag{6}$$

This decision and a completing tree and peeling schedule can be computed in polynomial time. A failed instance has a polynomial-size rank obstruction.

Proof. A successful tree meets each C_i by Corollary 3.2. Choose one of its edges in each block. The representatives are distinct because the blocks are disjoint, and they form a forest as a subset of a tree. This proves the necessary independent-transversal condition.

Conversely, a full independent transversal extends to a spanning tree of the connected graph D. That tree meets every C_i and hence succeeds. Apply Theorem 4.1 to decide and construct the transversal. Generate the actual schedule by peeling the full source matrix after deleting the resulting tree. If the answer is negative, the returned J violates (6). □

The criterion is stronger than the cardinality test $q \leq v - 1$. Different seed blocks may compete for the same graphical rank. For example, two singleton blocks represented by parallel dual edges have total rank one, so no tree can hit both even though each edge is individually eligible.

5.1 An optimum, not only a yes/no test

Definition 5.2 (Additional deletion deficit)**.** For a spanning tree T, permit an additional arbitrary set $Z \subseteq E \setminus T$ of row deletions before peeling, and put

$$\delta_D(N) = \min_{T \text{ tree}} \min\{|Z| : \mathcal{P}(T \cup Z) = E\}.$$

These additional deletions are a *formal measure of missing progress*. They are not automatically legal handle moves. A geometric algorithm wishing to use them must separately justify what deletes or changes those rows.

Theorem 5.3 (Exact pair-column deficit)**.** *For a pair-column input,*

$$\boxed{\delta_D(N) = q - \nu(C) = \max_{J \subseteq [q]} \left(|J| - r_D\left(\bigcup_{i \in J} C_i \right) \right).} \tag{7}$$

One can output an optimal tree T, an optimal extra-deletion set Z, a complete peeling schedule after $T \cup Z$, and a color subset J proving the lower bound, all in polynomial time.

Proof. For a fixed tree, every unhit source block needs a further deletion within itself; no deletion in a different component reaches it. Conversely, one row in each unhit block suffices. Thus its additional-deletion cost is exactly the number of blocks it misses.

The maximum number of blocks a tree can hit is $\nu(C)$. Any tree hitting h blocks contains an independent partial transversal of size h. Conversely, a maximum partial transversal extends to a tree hitting at least $\nu(C)$ blocks; it cannot hit another block without containing a larger independent partial transversal. This proves $\delta = q - \nu(C)$. Equation (5) gives the rank formula. Choose one extra row from each unhit block, peel the source matrix, and retain the final rank witness. □

A certificate consists of the tree, the extra rows, the pivots, and a color subset J for which $|J| - r_D(\bigcup_{i \in J} C_i) = |Z|$. The first three objects prove the upper bound; the rank equality proves the matching lower bound. In particular, a positive deficit has a meaningful interpretation even when the core is geometrically a handlebody: it measures the inadequacy of this raw presentation and these tree deletions, not the topology.

Example 5.4 (Zero rows and dual loops)**.** With two parallel dual edges and two zero matrix rows, a tree deletes only one row. The other is a stopping set, so $\delta = 1$. If an unanchored source block consists entirely of dual loops, its rank is zero and it forces positive deficit. The code retains both zero rows and loops; silently discarding either would invalidate the criterion.

6 A fixed-parameter extension for higher-support columns

6.1 The exceptional row boundary

Return to arbitrary N. Build G_N and A using only usable columns of support one or two. Let $\mathcal{H}$ be the remaining usable columns, and define

$$B = \bigcup_{h \in \mathcal{H}} \operatorname{supp}(h), \qquad b = |B|. \tag{8}$$

The union includes *all* rows of such a column, including its entries equal to two. A high-support column with no unit entry is not in $\mathcal{H}$ and does not contribute to B.

Call a source strongly connected component of G_N *mandatory* if it contains neither an anchor nor a row of B. Write these components as $U_1, \ldots, U_a$, and put

$$R_0 = \operatorname{Reach}\left(A \cup \bigcup_{j=1}^{a} U_j \right). \tag{9}$$

A mandatory component has no incoming low-column trigger and no high-column pivot at any of its rows. Every successful initial deletion set must therefore meet it. Once it has one seed, its whole component and all its low-column descendants are reached. This explains the symbolic use of the full U_j in (9); those rows are not being granted free unproved deletions.

For a row x, write

$$\mathrm{Anc}(x) = \{r \in E : x \in \mathrm{Reach}(\{r\})\}.$$

For each ordering $\pi = (x_1, \ldots, x_b)$ of B, process its prefixes $P_i = \{x_1, \ldots, x_{i-1}\}$. Declare x_i *automatic* at that prefix if either

$$x_i \in R_0 \cup \mathrm{Reach}(P_i), \tag{10}$$

or there exists $h \in \mathcal{H}$ with

$$n_{x_i h} = 1, \qquad \mathrm{supp}(h) \setminus \{x_i\} \subseteq P_i. \tag{11}$$

For every nonautomatic row, impose a special requirement

$$F \cap \mathrm{Anc}(x_i) \neq \varnothing. \tag{12}$$

There are at most b special requirements.

Lemma 6.1 (Separation of mandatory and special requirements)**.** *Every special candidate set* $\mathrm{Anc}(x_i)$ *is disjoint from* $\bigcup_j U_j$*.*

Proof. If a row of some U_j were an ancestor of x_i, then $x_i \in R_0$ and (10) would make it automatic. □

This disjointness is essential. It prevents one edge from having to count both as a mandatory representative and as a shared special witness in the reduction below.

6.2 Why ordering only exceptional rows is complete

Theorem 6.2 (Exceptional-order characterization)**.** *For an arbitrary matrix, a successful dual tree exists if and only if there is an ordering of* B *and a graphic-independent set* F *that meets every mandatory* U_j *and every special candidate set produced by that ordering.*

Sufficiency. Extend F to a tree if necessary. By monotonicity it is enough to reason with F. Its intersections with mandatory components, together with anchors, make all of R_0 available through low-column closure.

Induct on the order of B. If x_i satisfies (10), it follows from anchors, mandatory seeds, or earlier exceptional rows. If it satisfies (11), all other rows of that column are already deleted and its unit entry supplies a valid pivot unless x_i is already gone. Otherwise a seed in $F \cap \mathrm{Anc}(x_i)$ reaches it through low columns. Consequently every row of B is eventually deleted.

Every source component of G_N is now reached: it contains an anchor, is mandatory and seeded, or contains a row of B. Every component lies downstream of a source. Low-column closure therefore clears all remaining rows. As in Theorem 3.1, a column used earlier cannot still be needed to delete a new row, because all its other rows were already gone when it was used. □

Necessity. Suppose a tree T succeeds, and choose an actual complete peeling schedule after its rows are deleted. List the rows of $B \cap T$ first in arbitrary order, then the other rows of B in their deletion order. The resulting list is π. Every mandatory component must meet T, for the reason given above.

Consider a nonautomatic x_i. If it belongs to T, then $x_i \in \mathrm{Anc}(x_i)$ already supplies the required witness. If it was removed by a high-column pivot, all the other rows of that column lie in the earlier exceptional prefix, making (11) true, a contradiction. In the remaining case it was removed through low-column closure. Trace that closure backwards to an anchor, an initially deleted row, or a previously high-pivoted row. An anchor or a mandatory seed makes (10) true, as does an earlier high-pivoted row because that row lies in P_i. Therefore a nonautomatic x_i must have an ancestor in T. The tree itself is the required independent set F. □

The order is a sufficient-availability certificate, not a demand for exact physical timestamps. A seed may make a nominally later row available early. That can only help sufficiency; necessity uses the actual deletion order, so conservative prefix tests do not lose all valid orders.

6.3 Grouping witnesses rather than guessing their row names

For a fixed ordering, denote the special candidate sets by $Q_1, \dots, Q_k$, with $k \le b$. A single selected row may lie in several of these sets and satisfy all of their requirements. Requiring distinct representatives for every special set would be too strong.

Lemma 6.3 (Shared-witness partition reduction)**.** *Assume the disjoint mandatory sets $U_1, \dots, U_a$ are disjoint from every Q_i. There is a graphic-independent set hitting all mandatory and special sets if and only if some set partition Π of $[k]$ has a full independent transversal for the family*

$$U_1, \dots, U_a, \quad Q_K := \bigcap_{i \in K} Q_i \quad (K \in \Pi). \tag{13}$$

Proof. For a feasible independent set, choose one witness for each special requirement and group the indices which chose the same row. Distinct groups then have distinct representatives in their intersections. Choose a representative in each mandatory block. Mandatory representatives are distinct from each other and from every special witness, by the disjointness assumptions. All these representatives belong to the original independent set.

Conversely, a full independent transversal of (13) meets every mandatory block. A representative in Q_K simultaneously meets every special requirement indexed by K. Their union is the desired independent set. □

For example, two requirements with the same singleton candidate set $\{e\}$ need only one independent edge. The partition with one block captures this; forcing two differently colored clones of e would incorrectly demand a parallel-edge cycle. This is why simply invoking ordinary matroid intersection on all ancestor sets is not a correct general algorithm.

6.4 The algorithm and its bit bound

Theorem 6.4 (Fixed-parameter exact forest selection)**.** *Definition 2.1 can be decided in*

$$\mathrm{poly}(\Lambda)\, b!\, \mathrm{Bell}(b) = \mathrm{poly}(\Lambda)\, 2^{O(b \log(b+1))} \tag{14}$$

bit operations, where b is defined in (8). A positive answer has a tree-and-pivot certificate of polynomial size. A negative answer can be certified by at most $b!\,\mathrm{Bell}(b)$ independently checkable rank obstructions, with the corresponding branches regenerated from the source.

Proof. Construct the low digraph, its source components, and the exceptional boundary. For each of the $b!$ orders, generate its at most b special candidate sets. Enumerate their set partitions. For each partition, solve the independent- transversal instance in (13) using Theorem 4.1. Theorem 6.2 and Lemma 6.3 prove that a positive branch exists exactly when the original selection problem is positive. Extend its representatives to a tree and peel the full original matrix to produce the final certificate.

Mandatory blocks are disjoint, so they contribute at most m clones. There are at most b special partition blocks, each contributing at most m clones. Thus every branch has $P \leq m(b+1)$, and its intersection cost is polynomial by Proposition 4.3. Reachability, ancestor computation, and set intersection also have polynomial cost. Since $\mathrm{Bell}(k) \leq \mathrm{Bell}(b)$ for $k \leq b$, the number of branches is at most $b!\,\mathrm{Bell}(b)$. For $b \geq 1$, both $b!$ and $\mathrm{Bell}(b)$ are at most b^b, proving the exponential form; at $b = 0$ there is one branch.

Coefficient bits are read once and coefficients are saturated by Lemma 2.5. No arithmetic on expanded normal-sheet multiplicities is required by this finite core problem. Label manipulation adds polynomial bit factors. A failed branch returns a color subset whose rank is too small. Regenerating every order and every canonical partition and checking the supplied obstruction for each proves a negative answer without running an optimizer. □

Orders and partitions can be generated depth-first with polynomial workspace. The implementation can suppress the negative trace with `retain_negative=False`; the default stores it, and may therefore use $\mathrm{poly}(\Lambda)b!\,\mathrm{Bell}(b)$ output storage. A streaming proof writer would retain polynomial working memory while writing a potentially large negative proof. No polynomial-size negative certificate for all instances is claimed.

The parameter is the size of a higher-support *row boundary*, not the number of higher-support columns. One large usable column may make b large. Nor is merely bounding each column by three enough to make b small, as the next section shows.

7 A sharp abstract hardness boundary

Theorem 7.1 (NP-completeness at support three)**.** *The abstract problem in Definition 2.1 is NP-complete even under all of the following restrictions: every nonzero entry is one or two; every column has support at most three; and the dual graph is a rooted star with two parallel edges on each arm, together with loops at the root. The hard instances can have no zero rows and no singleton columns.*

Proof. Membership in NP follows from a spanning tree and a complete timestamp-checked peeling schedule. For hardness, reduce from Boolean satisfiability in conjunctive normal form, whose NP-completeness is the classical Cook–Levin foundation [9]. We may use instances with at least two nonempty clauses; harmless duplicated clauses give this restriction without affecting hardness.

For variable x_i, create two row edges t_i, f_i joining the root to a new vertex v_i. Every spanning tree chooses exactly one of them. Interpret these choices as the truth values. All other rows constructed below are root loops, so they can never be initial tree deletions.

An implication from previously deleted rows $a_1, \ldots, a_d$ to a target z is encoded by a column with entry one at z, entries two at the a_j, and zeros elsewhere. The target becomes peelable exactly after all prerequisites are deleted. No prerequisite can itself pivot at this column.

Create a loop row c_j for each clause. For every literal in that clause, add a one-prerequisite column from its corresponding literal row (t_i or f_i) to c_j. These columns implement the disjunction of the chosen literals. Combine all clause rows by a binary tree of two-prerequisite implication columns, creating new loop rows for its internal AND gates. Let z be its root, the accept row. Finally, add a one-prerequisite implication from z to every literal row. These cleanup columns allow the unchosen literal rows to disappear, but only after acceptance.

Every column has support two or three and entries in $\{0, 1, 2\}$. With nonempty clauses every clause row has a unit entry; every literal row has a unit cleanup entry; every AND row has a unit defining entry. Hence there are no zero rows. The number of rows and columns is polynomial in the formula length.

If a satisfying assignment is given, delete its selected literal tree rows. Every clause is then triggered, the AND tree evaluates to its accept row, and the cleanup columns remove all unchosen literals. All rows are deleted.

Conversely, consider the time at which the accept row first disappears in a successful peeling schedule. Before that time, an unchosen literal cannot have disappeared: its only columns in which it has entry one are cleanup columns requiring acceptance. Each deleted clause must therefore have had a literal chosen by the tree. Acceptance requires all leaves of the AND tree to have been deleted, so all clauses are satisfied by the tree's assignment. Thus the formula is satisfiable exactly when its matrix instance has a successful tree. Lemma 2.4 extends this equivalence from peeling to the permitted full positive-cancellation rule. □

The geometric limitation is substantive. Theorem 7.1 is not a hardness result for unknot recognition, for geometric handlebody recognition, or even for the class of source-derived raw cores in the repository. We have not realized these matrices and dual graphs by normal-cut attaching charts. Geometric compatibility may exclude the construction or force additional structure. What the theorem does show is that *the abstract incidence and dual-graph conditions alone* do not justify a polynomial selector for all bounded-arity inputs unless P equals NP.

The reduction also explains the FPT parameter: usable three-row columns form binary AND gates, while low columns transmit information through arbitrarily large directed networks. The exceptional boundary isolates where genuinely joint prerequisites occur; the rest is handled in polynomial time.

8 The native loop matrix, worked explicitly

The maintained source experiment gives five rows on the retained one-handle labels $(3, 5, 6, 7, 8)$ [3]:

$$N = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{pmatrix}. \tag{15}$$

There is one interior dual vertex. Rows $0, 1, 2, 3$ join it to the boundary root; row 4 is its loop. Thus the four singleton sets $\{0\}, \ldots, \{3\}$ are exactly the dual spanning trees.

The default choice deletes row 0. The remaining column totals are $(2, 3, 2, 3, 2)$, so all remaining rows form a stopping set. By Lemma 2.4, no change of positive-pivot order, including the use of fill, can rescue that particular tree choice.

The delivered general solver, with its incumbent shortcut disabled, finds the tree $\{1\}$ after 97 order/partition branches. Its positive certificate uses

$$(r, h) = (3, 3),\ (2, 8),\ (4, 5),\ (0, 6). \tag{16}$$

Here h is the original source column label, not its zero-based position in (15). To check the schedule, assign the tree row time -1 and the pivot rows their times $0, 1, 2, 3$:

Time	Pivot	Other source rows in its column	Their deletion times
0	$(3, 3)$	1	-1
1	$(2, 8)$	3	0
2	$(4, 5)$	$1, 2$	$-1, 1$
3	$(0, 6)$	$3, 4$	$0, 2$

Every pivot entry is one and every other incident row is earlier. One column, label 7, remains. This is a source-derived matrix-level recovery, not a new run of the original triangulation factory or Regina oracle.

The maintained experiment records another valid schedule for the same deleted row, using $(4, 7)$ in the third step instead of $(4, 5)$. That schedule leaves column 5 rather than column 7. This difference is not cosmetic when passive curves are present: a different surviving cocore can change a later private-port witness. Our native integration seam proposes the dual forest and then sends the source through its existing downstream checker rather than pretending that the local matrix certificate alone proves the whole cut.

9 Certificates, resource semantics, and integration

9.1 Positive packets

The local schema `dual-forest-positive-v1` contains a tree, its root-to-leaf dual cancellation order, and the one/two pivot pairs. The independent module `dualforest/verify.py` imports neither the producer, the dependency builder, nor the matroid optimizer. It checks the tree graph and the endpoint multiplicity of each dual move, then checks source timestamps for all pivots.

A row deleted by the tree has time -1. A pivot row has its listed time; every other source row meeting its pivot column must have smaller time. The checker also requires distinct pivot columns, exact row coverage, unit pivot entries, and valid labels. This costs one incidence scan plus graph and label bookkeeping. A malformed or incomplete proof is rejected. No topological verdict is emitted by these routines.

9.2 Optimality and negative packets

For pair-column instances, `pair-defect-v1` includes the optimal tree, additional seed rows, a finishing schedule, the claimed deficit, and obstruction colors. The independent checker rebuilds the low dependency relation from the source and computes its strongly connected blocks using a separate, simple reachability construction. It verifies the upper and lower bounds in Theorem 5.3. This verifier is polynomial, but its deliberately simple repeated traversals are not claimed to be linear.

For a general negative answer, `dual-forest-negative-v1` lists one rank obstruction per canonical branch. The checker regenerates all exceptional orders and all restricted-growth set partitions. Missing, extra, or invalid obstructions cause rejection. It performs no matroid-intersection search. Verification of a complete negative may be as expensive in the parameter as the search, which is consistent with the hardness boundary.

9.3 Unknown is distinct from no

The solver accepts a nonnegative `max_branches` cap. If it is exhausted before a positive branch or exhaustive negative completion, the answer is `unknown`, with no negative certificate. The cap counts intersection subproblems, not seconds or total instructions; input reading and dependency preparation have their own cost. A callback is polled at search and augmentation boundaries; its exceptions propagate, rather than becoming negative answers.

Python integers retain exactness. The source factory rejects Boolean values masquerading as integers, negative incidences, duplicate serialized columns, nonexistent endpoints, disconnected dual graphs, and inconsistent row counts. A test reads an incidence 2^{16384}, saturates it to two, and checks the expected answer. This checks the kernel's binary coefficient handling, not the size or cost of an entire normal-surface certificate with that multiplicity.

9.4 The native seam and what has not been executed

The record-level adapter maps native `None` endpoints to the root and each native three-handle ID to a distinct interior vertex. It preserves repeated endpoints and the original two-handle row IDs. Source one-handle column labels are preserved, including gaps. Four unit tests exercise the translation and malformed-record rejection.

The optional function `propose_geometry` in `integration/native_adapter.py` imports the pinned native producer, `_forest`, `_occurrences`, and `replay_core_three_handles`. It obtains the current endpoint records, checks them through the existing independent native source replayer, forms the fixed-primal-forest matrix, and searches for a useful tree. On local success it emits the existing native `core-three-handle-reduction-v1` shape and calls the native replayer again on that proposed forest. The returned geometry must still pass all normal core, port, boundary-pattern, and whole-cut verification.

Execution boundary: record translation was tested here; the native `propose_geometry` call was not. A full repository checkout and its native geometry dependencies were not available in the execution environment. No claim is made that the complete native test suite was rerun or that a production recognizer was modified. The archive is an integration candidate with an explicit source-replay gate, not a silently installed replacement.

Recommended rollout. Retain an already successful native route unchanged. Use the pair-column selector when its hypothesis holds. On a small number of dual choices, bounded literal trials may be cheaper than the general FPT kernel; the measurements below make that clear. Otherwise use the exceptional-boundary selector with an explicit budget and an exact fallback or inconclusive return. Record b, the number of source components, clone counts, actual branches, proof size, and complete-query timing. Only source replay and whole-query comparisons can justify a production default change.

10 Reproducible validation and measured tradeoffs

10.1 Executed checks

The package is standard-library Python, executed with Python 3.13.5. The 54 unit tests include directional incidence cases, loop obstructions, empty inputs, shared clone colors, an exchange requiring reassignment, exact deficits, source-timestamp mutation checks, missing negative branches, large binary entries, resource interruption, and native record translation.

The separate audit does not use the selector to supply its expected answers. Its small-instance oracle literally enumerates dual spanning trees and scans matrix column totals. The independent-transversal oracle enumerates color assignments and checks graphical independence directly. Deficit checks enumerate both trees and extra deletion subsets. The positive-fill oracle recursively tries all singleton positive pivots with the update (1).

Audit	Inputs	Mismatches
All three-row, two-column ternary matrices on all connected two-vertex dual multigraphs	13,851	0
All ternary 3×3 matrices on the loop graph	19,683	0
Mixed random matrices, $m \leq 10$, $b \leq 4$	2,000	0
Unrestricted random 5×5 matrices	150	0
Matroid solver versus assignment enumeration	1,000	0
Pair-column optimal deficits	500	0
Peeling versus all positive-pivot orders	19,683	0

Table 1: Executed finite audits. The pair-column stage also checks all 22,830 seed subsets of its 500 inputs. Counts in different stages are different comparison tasks, not a single disjoint population.

All 35,684 inputs in the first four rows receive exact answers, with zero mismatches and no branch-capped unknowns. Every positive certificate and every available negative certificate in that audit is replayed independently. The largest recorded branch count there is 3,220. These counts do not prove the asymptotic theorems; they audit the implementation against their finite consequences. Randomized tests are seeded and fully reproducible.

A separate hardness audit generates 400 CNF formulas and checks all 7,988 of their assignments against the corresponding tree deletions. Forty additional small formulas are compared with the exact general selector, and its positive or negative certificates are replayed. There are zero mismatches. These are checks of the explicit reduction, not experimental evidence of geometric realizability.

10.2 A family with exponentially many dual trees

For q arms, take two parallel edges a_i, b_i from the root to vertex i. Let row a_i have a private singleton unit column and let row b_i be zero. The graph has 2^q spanning trees. The sole successful one chooses every b_i, so that the anchored rows a_i remain for peeling. The pair-column selector identifies the singleton mandatory blocks $\{b_i\}$ and selects their independent representatives without enumerating trees.

With the enumeration order used in the benchmark, that successful tree is last. Swapping the row roles gives an easy-first control with a successful first tree. Adding a zero root-loop row makes every tree fail and gives deletion deficit one. These three families distinguish an asymptotic enumeration

obstruction from mere lucky or unlucky first choices. They are abstract matrices, not asserted normal-cut families.

10.3 Paired matrix-stage measurements

Both arms receive the same already parsed instance. One runs the pair-column selector; the other enumerates only *actual* spanning trees of this star family, not non-tree edge subsets, and applies literal peeling. Positive proof generation and replay are included. The pair solver additionally replays its optimality packet on negatives; the exhaustive negative baseline proves its answer by finishing its enumeration. Source construction and all native topology are excluded from both arms.

Each comparison has five randomly ordered pairs after an untimed warmup and three identical-selector/identical-selector control pairs. The repository matrix uses seven measurement pairs. All raw nanosecond-based samples, orders, and controls are retained in `results/benchmark.json`. A ratio above one favors the selector; the ratio is the median of paired ratios, not the quotient of medians.

Input	Selector (ms)	Trees (ms)	Ratio	A/A
Unique last, $q = 1$	0.039	0.016	0.398	0.967
Unique last, $q = 4$	0.068	0.098	1.375	0.966
Unique last, $q = 8$	0.135	3.697	28.222	0.941
Unique last, $q = 12$	0.250	136.882	553.569	1.057
Easy first, $q = 12$	0.181	0.074	0.403	0.736
Obstructed, $q = 12$	0.359	134.829	381.331	0.939
Repository loop matrix	1.062	0.032	0.031	1.029

Table 2: Matrix-stage medians and paired ratios, not native recognition timings. The loop row uses the general FPT kernel; the star rows use the polynomial pair-column kernel. A/A compares two identical selector executions.

On the unique-last $q = 12$ family the measured paired ratio is approximately 554, and the obstructed family gives approximately 381. These large factors come from avoiding 4,096 enumerated trees in a deliberately specified structural family. They are *not* speedup claims for a knot recognizer.

The negative results matter equally. The easy-first $q = 12$ baseline is faster, with paired ratio about 0.403. For the five-row repository matrix the general FPT selector has ratio about 0.0306: literal tree enumeration is roughly 32.7 times faster on this tiny example. The selector solves a scalable structural problem, but its first implementation should not displace inexpensive bounded trials. The matrix example demonstrates recovered local coverage, not a wall-clock improvement in native recognition.

11 What this proves toward quasi-polynomial recognition

Let a correct surrounding algorithm make $Q(n)$ authenticated core-selection queries on an n-crossing input. For query i, let Λ_i be its actual encoded input length and b_i its exceptional row boundary. Let $C_{\mathrm{other}}(n)$ charge every other task, including useful surface discovery, normal-coordinate and pattern transport, source authentication, geometric rebuilding, reset work, and any fallback outside this selector.

Theorem 11.1 (Explicit composition budget)**.** *If the surrounding algorithm needs exactly the local decision in Definition 2.1, and its correctness is maintained by the source and downstream checks described above, then replacing its forest selection by Theorem 6.4 gives total cost at most*

$$C_{\mathrm{other}}(n) + \sum_{i=1}^{Q(n)} \mathrm{poly}(\Lambda_i)\, 2^{O(b_i \log(b_i+1))}. \tag{17}$$

In particular, the target $n^{O(\log n)}$ follows from the sufficient conditions

$$C_{\mathrm{other}}(n),\ Q(n) \le 2^{O((\log n)^2)}, \tag{18}$$

$$\Lambda_i \le n^{O(1)}, \tag{19}$$

$$b_i \log(b_i + 1) = O((\log n)^2) \quad \textit{uniformly in } i. \tag{20}$$

For example, $b_i = O((\log n)^2/\log\log n)$ suffices for large n.

Proof. Each query cost is Theorem 6.4; all omitted work is charged to C_{other}. Under the stated conditions each summand and the number of summands are bounded by $2^{O((\log n)^2)}$. Their product has the same form. The example for b_i follows from $\log(b_i + 1) = O(\log\log n)$. □

This theorem is an accounting implication, not a proof of its hypotheses. No universal bound on the realized b_i, source sizes, or reset cost is established here. In particular, a local successful tree may leave a different passive disk system and alter later costs. Neither the pair-column theorem nor the hardness construction resolves that geometric issue.

The iteration bound $L(g+1)^L$ in Section 9 of Lackenby's preprint [7] highlights why local search cost and hierarchy rebuild cost must be kept separate. The new selector removes spanning-tree enumeration in its polynomial case and confines its own nonpolynomial work to an explicit higher-support row boundary. It does not by itself control L, g, or the costs of the other operations.

12 Further research questions and concrete next steps

Research question 12.1 (Measure the source boundary, not only tetrahedra)**.** What are the distributions of b, low-graph source components, and graphical rank deficits on the maintained authenticated cut corpus, including historical source cores and the later loop/interior-vertex fixtures?

The next experiment should export the raw counts and dual endpoints from the native source checker, then record both kernel time and completed whole-cut time. The historical native corpus was not rerun in this package. A useful outcome would identify whether difficult cases have a small exceptional boundary or whether a different representation is needed.

Research question 12.2 (Choose the primal forest to reduce the exceptional boundary)**.** Can one optimize or approximate b over authenticated zero/one-handle forests, while keeping the dual endpoints and the later port data source-bound?

The current theory deliberately freezes that forest. A primal change can remove columns and alter which columns have high support; treating those changes as free preprocessing would conceal a second search. A useful theorem would combine primal selection with a controlled, explicitly charged change in the row boundary.

Research question 12.3 (Compress exceptional orders)**.** Can the $b!\,\mathrm{Bell}(b)$ factor be replaced by $2^{O(b)}$, perhaps through a representative-family or subset dynamic program that preserves shared witnesses?

The obstruction is not merely enumerating permutations: multiple ancestor requirements may share one edge, and the chosen edges must remain jointly independent with many mandatory blocks. A correct compression must preserve those simultaneous constraints, not only whether each demand has some candidate.

Research question 12.4 (Parameterize high targets rather than all high-support rows)**.** Can one replace b by the number of rows which carry a unit entry in a higher-support column, plus a smaller interface parameter for its nonunit rows?

A nonunit row can never be the high-column pivot, but its deletion is still a prerequisite. The present proof includes it in B so that the prefix test is complete. Omitting it without a substitute would invalidate the order theorem. A succinct representation of these prerequisites could be especially valuable for large sparse geometric interfaces.

Research question 12.5 (What restrictions does genuine geometry impose?)**.** Which matrix-and-dual-graph pairs are realizable by the authenticated source core construction, and do those restrictions exclude the satisfiability gadgets of Theorem 7.1?

A realization theorem or an obstruction theorem would both be useful. Until one is proved, the abstract NP-completeness result must not be promoted to a hardness statement about normal cuts. Conversely, dismissing it without proving an appropriate source restriction would not justify a general polynomial selector.

Research question 12.6 (Turn deletion deficits into legal geometric progress)**.** Given an optimal deficit certificate, can the indicated unhit source blocks be resolved by a bounded-cost collection of authenticated disk changes or local moves, with a provable decrease in a global progress measure?

The additional rows Z are not legal deletions merely because the optimizer names them. A useful bridge would associate a geometric operation and an explicit cost or obstruction with each rank-deficient group, rather than using the deficit as an informal heuristic.

Research question 12.7 (Optimize downstream witnesses, not just raw completion)**.** Among successful dual forests, can one select a forest whose resulting cocore system preserves enough private ports or pattern witnesses to minimize later rebuilding?

The loop example already has different valid pivot schedules leaving different columns. A weighted representative objective may help when costs are additive, but passive-witness compatibility need not be additive or matroidal. A sound state description and source-bound transport theorem are required before a weighted optimization can claim a hierarchy benefit.

Research question 12.8 (Count-aware or bounded-trial portfolios)**.** Can a cheap bound on the number of dual trees choose reliably between bounded literal enumeration and the FPT kernel, while preserving a clean worst-case budget?

The benchmark suggests this immediately: the small native example favors literal trials, while the star families favor matroid selection. A fixed number of verified trials is easy to charge before the exact selector. More ambitious selection could use an exact tree count and an output-sensitive tree enumerator; those components would need their own bit and certificate accounting.

Research question 12.9 (Stream and compress negative branch proofs)**.** Can identical rank obstructions across different orders and witness partitions be shared without requiring the checker to trust a producer's branch omissions?

The current negative checker deliberately regenerates every branch. A compressed proof would need a verifiable coverage argument for the omitted order regions. This may lead to practical reductions well before a sharper theoretical FPT bound.

Research question 12.10 (Establish a source-aware quasi-polynomial envelope)**.** Can the hierarchy construction be arranged so that the actual source sizes, exceptional boundaries, and number of rebuilds satisfy (18)–(20) simultaneously?

The simultaneous qualification matters. A representation reducing b may enlarge the source or disrupt a previously short hierarchy. A theorem controlling only one quantity does not close the recognition bound. The most consequential next step is a geometric normal form that preserves source authentication while controlling all three quantities over the entire run.

13 Conclusion

A complete dual cancellation forest is not necessarily a useful one. For the pair-column regime, the correct missing condition is an independent transversal of unanchored source components, and this gives both a polynomial selector and an exact deletion-deficit certificate. For general cores, the higher-support row boundary is enough to localize the nonpolynomial search: exceptional orders and shared-witness partitions reduce every branch to the same checkable matroid primitive. The support-three hardness construction shows why a general abstract polynomial claim would require more structure.

The delivered kernel recovers the documented matrix-level loop miss and passes its finite audits, while the benchmark honestly retains cases where direct tree trials are faster. Its immediate role is an opt-in, source-replayed selection component and a measurable structural parameter for the maintained pipeline. The route to unrestricted quasi-polynomial recognition still needs a theorem controlling the source geometry, exceptional boundaries, and hierarchy resets jointly.

A Algorithm pseudocode and certificate interfaces

A.1 The pair-column selector

```
G, anchors = low_column_dependencies(N)
blocks = source_SCCs(G) without anchors
I, rank_witness = maximum_independent_transversal(D, blocks)
T = extend_to_spanning_tree(D, I)
Z = one row from each block not met by T
pivots = peel(N, initially_deleted=T | Z)
return T, Z, pivots, rank_witness
```

When Z is empty this is a positive selection proof. Otherwise the rank witness and complete schedule after $T \cup Z$ prove the optimum deletion deficit.

A.2 The general selector

```
G, anchors, high_columns, B = classify_usable_columns(N)
U = source_SCCs(G) containing neither an anchor nor a row of B
R0 = reach(G, anchors | union(U))
for order in permutations(B):
    Q = nonautomatic_ancestor_requirements(order, R0, G, high_columns)
    for partition in set_partitions(indices(Q)):
        groups = U + [intersection(Q[i] for i in block)
                      for block in partition]
        I, obstruction = maximum_independent_transversal(D, groups)
        if I represents every group:
            T = extend_to_spanning_tree(D, I)
            return YES, T, peel(N, T)
        record(obstruction)
return NO, complete_branch_obstruction_record
```

A cap checked before each intersection call replaces the last line by `UNKNOWN` when exhausted. No bound on source preparation is implied by that branch cap.

A.3 Reproduction commands

From the extracted package root:

```
python -m unittest discover -s tests -v
python -m experiments.validate
python -m experiments.sat_audit
python -m experiments.benchmark
python -m dualforest examples/repository_loop.json --max-branches 10000
sh article/build.sh
```

The example command consumes a normalized matrix/dual-graph input, not a knot diagram. It cannot certify unknotness by itself. The JSON packet explicitly records its local scope.

B Source and artifact provenance

The primary source pin is `c6d0ec8aa0b6308310ab6c081ba29d8bc363ede5`. The mathematical source readings were the full maintained peeling, dual-three-handle, and dual-result sections; the native three-handle producer and independent checker were also read in full. The native `core_handlebody.py` reading covered its input contract, primal forest, occurrence extraction, and the beginning of its capped planner. An indexed search confirmed the current column-peeling entry point, but the whole native module was not imported or executed.

The incoming directory was inventoried. In particular, `ProveIt_Dual_Forest_Normalization_2026-10-12.zip` and `ProveIt_Dual_Forests_Homological_Progress_2026-10-12.zip` were present, but their internal manuscripts and code were not read. The present report must be reconciled with those deliveries during intake before any claim of repository-wide novelty. No incoming files or maintained repository files were changed by this session.

The included examples, source-neutral implementation, synthetic reduction, finite-oracle code, recorded outputs, and this article were generated for the present package. The five-row matrix is

explicitly attributed to the maintained source result. External research papers and Lackenby's slide deck are cited but not redistributed. `README.md` gives the file map and integration instructions; `CLAIMS.md` separates proven, tested, inherited, conditional, and unexecuted items.

References

[1] V. Reshetnikov and contributors. *Complete rectangular cancellation by sparse column peeling*. ProveIt, `Topology/UnknotRecognition/synthesis/column_cancellation.tex`, commit `c6d0ec8aa0b6308310ab6c081ba29d8bc363ede5`, 2026. https://github.com/VladimirReshetnikov/ProveIt.

[2] V. Reshetnikov and contributors. *Removing source core three-handles relative to the bundle ports*. ProveIt, `Topology/UnknotRecognition/synthesis/core_three_handles.tex`, same pinned commit, 2026.

[3] V. Reshetnikov and contributors. *Source tests, dual-forest dependence, and three-handle results*. ProveIt, `Topology/UnknotRecognition/synthesis/three_handles_results.tex`, same pinned commit, 2026. The five-row loop matrix and original alternative certificates are recorded here.

[4] V. Reshetnikov and contributors. *Source-authenticated removal of core three-handles by a dual boundary forest*. ProveIt, `Topology/UnknotRecognition/fast/fastunknot/core_three_handle_reduction.py`, same pinned commit, 2026.

[5] V. Reshetnikov and contributors. *Independent replay of the source core's dual two/three cancellation forest*. ProveIt, `Topology/UnknotRecognition/fast/fastunknot/core_three_handle_reduction_verify.py`, same pinned commit, 2026.

[6] M. Lackenby. *Quasi-polynomial unknot-recognition lecture slides*. Oxford-hosted slide deck, inspected 12 October 2026; displayed main theorem at PDF page 13. https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf.

[7] M. Lackenby. *Incompressible surfaces, hierarchies and unknot recognition*. arXiv:2607.23350v1, 2026, especially Section 9 and Proposition 9.1. https://arxiv.org/html/2607.23350v1.

[8] M. Bárász, K. Bérczi, T. Király, Y. Yamaguchi, and Y. Yokoi. *Matroid Intersection under Minimum Rank Oracle*. arXiv:2407.03229v1; EGRES Technical Report TR-2024-13, 2024. The version is specified because later versions have changed author metadata. https://arxiv.org/html/2407.03229v1.

[9] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971. https://doi.org/10.1145/800157.805047.